CHAPTERS 41–45

Security & Deployment

All of Part VII (Security) and the start of Part VIII (Infrastructure & Operations).

Chapters 41–44 secure the system; Chapter 45 begins how we deploy and operate it.

CHAPTER 41

Authentication & Sessions

Part VII · Security — proving who a user is.

LEARNING OBJECTIVES

- Distinguish authentication from authorization.
- Explain secure password storage (salted, slow hashing).
- Compare session-based and token-based (JWT) authentication.
- Explain OAuth/OIDC single sign-on and multi-factor authentication.

REAL-WORLD ANALOGY — ID AND A VISITOR BADGE

To enter a secure building you show **ID** once at reception (your credentials), and receive a **visitor badge** you flash at each door afterward — you don't re-prove your identity every time. Authentication is that ID check; the session or token is the badge you carry for the rest of your visit.

41.1 The problem

Authentication answers "are you who you say you are?" — distinct from **authorization** ("what may you do?", Chapter 42). And because HTTP is **stateless** (Chapters 4, 6), the server needs a way to remember you're logged in across requests.

41.2 Password security

NEVER STORE PASSWORDS IN PLAINTEXT — OR EVEN ENCRYPTED

Store a **salted hash** using a deliberately **slow** algorithm (bcrypt, scrypt, or Argon2). The **salt** (a unique random value per password) defeats precomputed rainbow tables; the **slowness** makes brute-forcing expensive. Hashing is one-way, so a breach doesn't reveal the passwords. Never *encrypt* passwords (reversible) — hash them.

41.3 Sessions vs tokens

Two ways to remember a logged-in user — the stateful-vs-stateless choice from Chapter 6:

sessions store state server-side; tokens carry signed claims — easier to scale, harder to revoke

Session-based: the server stores session data and gives the client a session-ID cookie; every request is looked up. It's **stateful**, so horizontal scaling needs a shared session store (Redis, Chapters 6/8). **Token-based (JWT):** the server issues a **signed** token holding the user's claims; the client sends it each request and the server just verifies the signature — **stateless** and easy to scale, but harder to revoke, so keep tokens short-lived with refresh tokens.

41.4 OAuth, OIDC, and SSO

OAuth 2.0 lets a user grant your app limited access to their account elsewhere without sharing a password — the basis of "Log in with Google." **OpenID Connect** adds an identity layer on top for authentication, enabling **single sign-on (SSO)** across services.

41.5 Multi-factor authentication

MFA combines factors — something you **know** (password), **have** (a phone/TOTP or hardware key), or **are** (biometric). A stolen password alone is then useless, making MFA one of the highest-value defenses.

Common mistakes

WATCH OUT

- Storing passwords in plaintext or with fast/unsalted hashes.
- Putting sessions in one server's memory, breaking horizontal scaling.
- Long-lived JWTs you can't revoke; storing sensitive data in a (readable) JWT.
- Confusing authentication with authorization.
- Skipping MFA on high-value accounts.

Interview questions

1. Authentication vs authorization?
2. How should passwords be stored, and why salted + slow?
3. Compare session-based and JWT authentication.
4. What do OAuth and OIDC provide?
5. Why does MFA matter, and what are the factors?

Hands-on exercises

1. Hash a password with bcrypt and explain the salt and cost factor.
2. Decide sessions or JWT for a horizontally-scaled API, and justify.
3. Describe how to revoke a compromised JWT despite statelessness.
4. Add a TOTP second factor to a login flow (conceptually).

Mini project

BUILD A LOGIN TWO WAYS

Implement login with salted bcrypt password hashing, then support it via (1) server sessions backed by Redis and (2) short-lived JWTs plus refresh tokens. Compare how each behaves when you run two server instances, and implement JWT revocation via a short expiry and a refresh flow.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE AUTH SYSTEM

Design authentication: bcrypt/Argon2 password storage, JWT access tokens with refresh tokens (stateless to fit your scaled app tier), “Log in with Google” via OAuth/OIDC, and MFA for sensitive actions. Decide token lifetimes and your revocation strategy.

Chapter summary

- **Authentication** (“who are you?”) is distinct from authorization.
- Store passwords as **salted, slow hashes** (bcrypt/Argon2) — never plaintext or encrypted.
- **Sessions** are stateful (need a shared store); **JWTs** are stateless and scalable but harder to revoke.
- **OAuth/OIDC** enable delegated login and SSO; **MFA** defeats stolen passwords.

CHAPTER 42

Authorization

Part VII · Security — deciding what a user may do.

LEARNING OBJECTIVES

- Distinguish authorization from authentication.
- Compare ACLs, RBAC, and ABAC.
- Apply least privilege and deny-by-default.
- Explain where to enforce authorization and the broken-access-control risk.

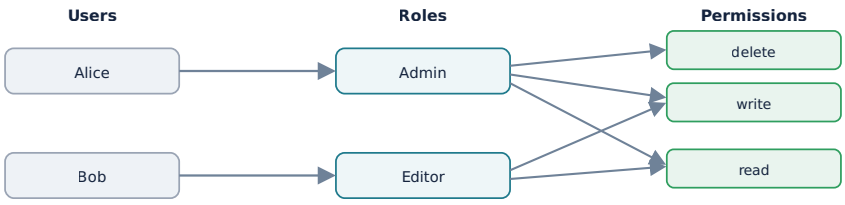
REAL-WORLD ANALOGY — WHAT YOUR BADGE OPENS

Everyone in the building has a badge (they're authenticated), but each badge opens only certain doors. A cleaner's badge won't open the server room; an engineer's won't open the finance vault. **Authorization** is the rule set that decides which doors each badge opens.

42.1 The problem

Once a user is authenticated (Chapter 41), the system must decide what they're **allowed** to do: "can this user perform this action on this resource?" That's **authorization** — access control.

42.2 Access-control models



RBAC: users get roles; roles carry permissions — scalable and easy to reason about

Model	How it works
ACL	Per-resource list of who can do what. Fine-grained but hard to manage at scale.
RBAC	Users get **roles**; roles carry **permissions**. Scalable and common.
ABAC	Decisions from **attributes** (user, resource, context) via policies. Flexible but complex.

42.3 Least privilege and deny by default

Grant the **minimum** permissions each user or service needs, and **deny by default** — access is refused unless explicitly allowed. This shrinks the blast radius if an account is compromised.

42.4 Where to enforce it

KEY INSIGHT — AUTHENTICATE ONCE, AUTHORIZE EVERY ACTION

Enforce authorization on **every** request — at the gateway (Chapter 9) and/or in each service — and **never trust the client**. The most common serious vulnerability (OWASP's top item) is **broken access control**: missing or wrong checks. A classic case is **IDOR** — changing an ID in a URL (`/orders/123` → `/orders/124`) to read someone else's data because the server never verified ownership. Centralizing policy (e.g. a policy engine) helps keep checks consistent.

Common mistakes

WATCH OUT

- Checking authentication but forgetting authorization on an endpoint.
- Trusting client-supplied roles/IDs instead of verifying server-side (IDOR).
- Over-broad roles that violate least privilege.
- Scattered, inconsistent checks instead of centralized policy.

Interview questions

1. Authorization vs authentication?
2. Compare ACL, RBAC, and ABAC.
3. What is least privilege and deny-by-default?
4. What is broken access control / IDOR, and how do you prevent it?
5. Where should authorization be enforced?

Hands-on exercises

1. Design RBAC roles and permissions for a blog (admin, author, reader).
2. Show how an IDOR bug lets a user read another's order, and fix it.
3. Decide RBAC or ABAC for a system where access depends on time and location.
4. List which endpoints need authorization checks in a simple API.

Mini project

ADD RBAC AND BREAK IDOR

Implement RBAC in a small API (roles → permissions). Then deliberately introduce an IDOR bug (fetch a record by ID without an ownership check), exploit it, and fix it by verifying the resource belongs to the requester. Confirm every mutating endpoint enforces authorization.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — ACCESS CONTROL

Define roles (user, moderator, admin) and permissions for posts, comments, and moderation. Enforce ownership checks (a user edits only their own posts) to prevent IDOR, apply least privilege to service accounts, and decide where checks live (gateway vs service). Deny by default everywhere.

Chapter summary

- **Authorization** decides what an authenticated user may do.
- **ACL** (per-resource), **RBAC** (roles→permissions, common), **ABAC** (attribute policies).
- Apply **least privilege** and **deny by default**.
- Enforce on every request; **broken access control** (e.g. IDOR) is the top web vulnerability.

CHAPTER 43

Encryption, TLS & PKI

Part VII · Security — protecting data in transit and at rest.

LEARNING OBJECTIVES

- Compare symmetric and asymmetric encryption.
- Distinguish hashing from encryption and explain digital signatures.
- Explain how TLS combines both, and how certificates/PKI establish trust.
- Explain encryption at rest and key management.

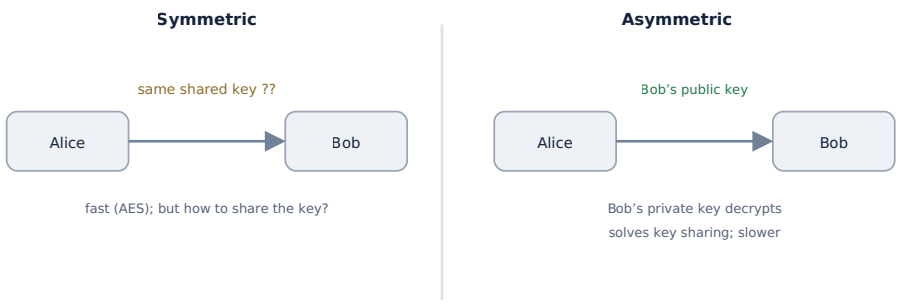
REAL-WORLD ANALOGY — LOCKS, KEYS, AND A NOTARY

A **symmetric** lock uses one key that both locks and unlocks — fast, but both parties must share that key secretly. An **asymmetric** lock has two keys: a public one anyone can use to lock a box, and a private one only you hold to open it. And a trusted **notary** (a certificate authority) vouches that a public key really belongs to whom it claims. Those three ideas underpin all of TLS.

43.1 The problem

We must protect data's **confidentiality** and **integrity** — both **in transit** (HTTPS, Chapter 4) and **at rest** (stored). This chapter is the cryptography beneath that.

43.2 Symmetric vs asymmetric



Symmetric encryption (AES) uses one shared key — fast, but the parties must exchange the key securely. **Asymmetric** (public-key) encryption uses a key pair: encrypt with the public key, decrypt with the private one — solving key distribution, but slower.

43.3 Hashing vs encryption; signatures

Hashing is one-way (for integrity and password storage, Chapter 41); **encryption** is reversible (for confidentiality). A **digital signature** combines them — hash the data, encrypt the hash with your private key — proving **authenticity and integrity**: anyone with your public key can verify it was you and that nothing changed.

43.4 How TLS combines both

KEY INSIGHT — ASYMMETRIC TO BOOTSTRAP, SYMMETRIC FOR SPEED

TLS (Chapter 4) uses **asymmetric** crypto in the handshake to authenticate the server and agree on a shared **symmetric** session key, then switches to fast **symmetric** encryption for the actual data. You get the trust of public-key crypto and the speed of symmetric — the best of both.

43.5 Certificates and PKI

How do you trust a server's public key? A **certificate** binds a public key to an identity, signed by a **Certificate Authority (CA)**. Browsers trust a set of root CAs and follow the **chain of trust** down to the site's certificate. This **Public Key Infrastructure (PKI)** is what makes the padlock meaningful; Let's Encrypt issues such certificates free.

43.6 Encryption at rest

Encrypt stored data — disks, databases, backups — so a stolen disk is useless. The hard part is **key management**: use a managed **key service (KMS)**, rotate keys, and use **envelope encryption** (a data key encrypted by a master key). And the golden rule: **never roll your own crypto** — use vetted libraries.

Common mistakes

WATCH OUT

- Implementing custom crypto instead of standard, audited libraries.
- Confusing hashing (one-way) with encryption (reversible).
- Shipping data unencrypted at rest, or hardcoding/committing keys.
- No key rotation or management strategy.
- Trusting a certificate without validating the chain.

Interview questions

1. Compare symmetric and asymmetric encryption and their trade-offs.
2. How does TLS use both?
3. Hashing vs encryption; what is a digital signature?
4. How do certificates and CAs establish trust?
5. How do you handle encryption at rest and key management?

Hands-on exercises

1. Explain why TLS doesn't just use asymmetric encryption for everything.
2. Trace the chain of trust from a root CA to a website certificate.
3. Decide hashing vs encryption for: passwords, credit-card numbers, file integrity.
4. Describe envelope encryption with a KMS.

Mini project

ENCRYPT IN TRANSIT AND AT REST

Serve an app over HTTPS with a real certificate (e.g. Let’s Encrypt) and inspect the certificate chain. Then encrypt a sensitive field at rest using a data key wrapped by a master key (envelope encryption), and demonstrate rotating the master key without re-encrypting all data.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE CRYPTO PLAN

Specify: TLS everywhere (edge and, via the mesh, service-to-service, Chapter 30); passwords hashed (Chapter 41); sensitive fields (emails, tokens) encrypted at rest with a KMS and rotation; and signed tokens/webhooks. Note where you rely on symmetric vs asymmetric and confirm no custom crypto.

Chapter summary

- **Symmetric** (one shared key, fast) vs **asymmetric** (key pair, solves key distribution, slower).
- **Hashing** is one-way; **encryption** reversible; **signatures** prove authenticity + integrity.
- **TLS** uses asymmetric to agree a symmetric session key; **certificates/PKI** establish trust via CAs.
- Encrypt **at rest** with managed keys and rotation; never roll your own crypto.

CHAPTER 44

Common Attacks & Defenses

Part VII · Security — the threats every system faces.

LEARNING OBJECTIVES

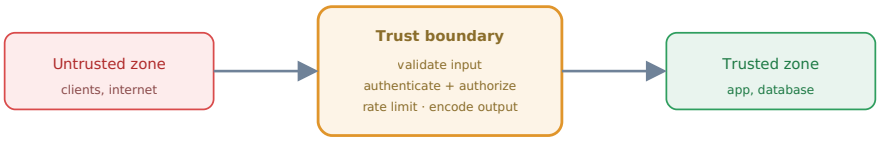
- Explain injection, XSS, CSRF, and SSRF, and their defenses.
- Explain how DDoS is mitigated.
- Apply the core principles: validate input, encode output, least privilege, defense in depth.
- Manage secrets and dependencies safely.

REAL-WORLD ANALOGY — KNOWING THE BURGLAR'S TRICKS

You secure a house best by knowing how burglars actually get in — the unlocked back window, the spare key under the mat, the tailgater at the gate. Application security is the same: learn the handful of common attacks and put the right lock on each entry point.

44.1 The problem

Systems face constant attacks. A small set of vulnerability classes (the spirit of the OWASP Top 10) accounts for most real breaches. Security is a first-class design concern, and the unifying rule is simple: **never trust input**.



everything crossing the boundary is checked — never trust the client

44.2 The common attacks and their defenses

Attack	What it is	Defense
Injection (SQLi)	Untrusted input executed as a query/command.	**Parameterized queries**; never concatenate input; least-privilege DB user.
XSS	Malicious script injected into a page, runs in victims' browsers.	**Encode/escape output**; Content-Security-Policy; sanitize input.
CSRF	A logged-in user's browser tricked into an unwanted request.	**CSRF tokens**; **SameSite** cookies.
SSRF	Server tricked into requesting internal resources.	**Allowlist** outbound URLs; block internal ranges.
DDoS	Flood of traffic to exhaust resources.	Rate limiting (Ch35), CDN/scrubbing, autoscaling, a WAF.

44.3 Secrets and dependencies

Never hardcode or commit **secrets** (API keys, DB passwords) — use a secrets manager/vault and inject them at runtime. Keep **dependencies patched**: known-vulnerable libraries are a leading breach cause, so scan and update them. Also avoid **security misconfiguration** (default passwords, open buckets, verbose errors).

44.4 The principles

KEY INSIGHT — A FEW RULES STOP MOST ATTACKS

Validate all input, **encode all output**, apply **least privilege** (Chapter 42), **encrypt everything** (Chapter 43), **patch dependencies**, and practice **defense in depth** (assume any one control can fail). Most breaches trace back to violating one of these basics — not to exotic attacks.

Common mistakes

WATCH OUT

- Building SQL by string concatenation instead of parameterized queries.
- Rendering user content without output encoding (XSS).
- State-changing GETs and no CSRF protection.
- Hardcoded secrets in code or config committed to version control.
- Ignoring dependency vulnerabilities and default configs.

Interview questions

1. How do SQL injection and XSS work, and how do you prevent each?
2. What are CSRF and SSRF, and their defenses?
3. How is a DDoS attack mitigated?
4. How should secrets be managed?
5. What core principles prevent most vulnerabilities?

Hands-on exercises

1. Rewrite a concatenated SQL query as a parameterized one.
2. Show an XSS payload and the output-encoding that stops it.
3. Add SameSite + CSRF-token protection to a form.
4. Move a hardcoded API key into a secrets manager.

Mini project

ATTACK, THEN DEFEND

Build a tiny vulnerable app and demonstrate SQL injection and reflected XSS against it. Then fix each: parameterized queries, output encoding plus a CSP, and SameSite/CSRF

tokens on forms. Move any secrets into environment/vault injection and add a dependency vulnerability scan.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE THREAT MODEL

Write a threat model: parameterized queries everywhere; output encoding + CSP for user-generated posts (a prime XSS target); CSRF protection on state-changing endpoints; SSRF allowlists for any URL-fetching feature (link previews); DDoS defense via CDN + rate limits (Chapter 35); secrets in a vault; and automated dependency scanning.

Chapter summary

- **Injection** → parameterized queries; **XSS** → output encoding + CSP; **CSRF** → tokens + SameSite; **SSRF** → URL allowlists; **DDoS** → rate limits + CDN.
- Keep **secrets** out of code (use a vault) and **dependencies** patched.
- The unifying rules: **validate input, encode output, least privilege, encrypt, patch, defense in depth**.
- **Never trust input** — check everything crossing the trust boundary.

COMING NEXT — PART VIII

Chapter 45 — Containers & Orchestration opens Part VIII (Infrastructure & Operations): how modern applications are packaged into containers and run at scale by orchestrators like Kubernetes.

CHAPTER 45

Containers & Orchestration

Part VIII · Infrastructure & Operations — how modern apps are packaged and run.

LEARNING OBJECTIVES

- Explain the “works on my machine” problem and how containers solve it.
- Compare virtual machines and containers.
- Explain what Docker images provide.
- Explain what an orchestrator like Kubernetes does.

REAL-WORLD ANALOGY — SHIPPING CONTAINERS

Before standardized shipping containers, loading a ship meant hand-packing odd-shaped cargo. The container changed everything: a sealed, standard box that any ship, truck, or crane handles identically. Software **containers** do the same — a standard unit that runs the same on a laptop, a server, or the cloud — and an **orchestrator** is the port operator scheduling thousands of them.

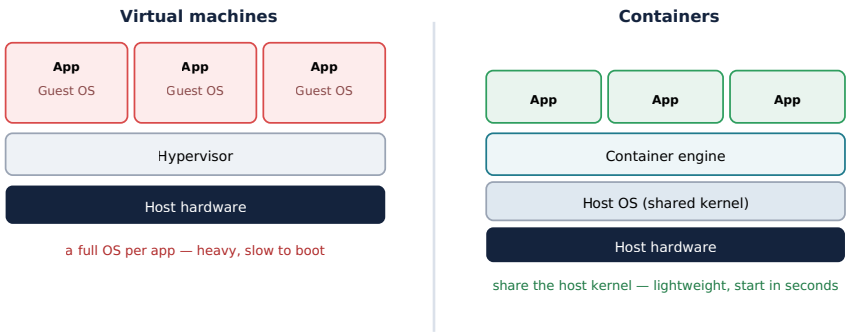
45.1 Part VIII begins

We’ve designed and secured the system; now we **deploy and operate** it. Part VIII covers containers, CI/CD, infrastructure as code, cloud architecture, observability, and capacity planning. It starts with how applications are packaged and run.

45.2 The problem

“It works on my machine” — software behaves differently across environments because of mismatched OS versions, libraries, and configuration. We need a way to package an app **with its entire environment** so it runs identically everywhere.

45.3 Virtual machines vs containers



A **virtual machine** virtualizes hardware and runs a full **guest OS** per app — strong isolation, but heavy (gigabytes, boots in minutes). A **container** virtualizes the OS, sharing the host **kernel** — lightweight (megabytes, starts in seconds), so you can pack far more per machine.

45.4 Docker and images

Docker packages an app plus its dependencies into an **image** — a portable, immutable artifact built from layers and stored in a registry. Run the image and you get an identical container anywhere, ending environment drift.

45.5 Orchestration with Kubernetes

KEY INSIGHT — DECLARE THE DESIRED STATE; THE ORCHESTRATOR MAINTAINS IT

Running many containers across many machines needs scheduling, scaling, health checks, networking, and recovery. **Kubernetes** is the orchestrator: you declare the **desired state** ("run 5 replicas of this service") and it continuously makes reality match — scheduling **pods**, load-balancing via **services**, **auto-scaling** (Chapter 10), and **self-healing** by restarting failed containers. It builds on ideas you've met: load balancing (Chapter 7) and service discovery (Chapter 30).

Common mistakes

WATCH OUT

- Treating containers as tiny VMs (long-lived, mutable) instead of immutable, disposable units.
- Putting persistent state inside a container's writable layer.
- Adopting Kubernetes' full complexity for a tiny system that doesn't need it.
- Huge images with unnecessary layers and no security scanning.

Interview questions

1. What problem do containers solve?
2. Compare VMs and containers.
3. What is a Docker image, and why is immutability useful?
4. What does an orchestrator like Kubernetes do?
5. How does declarative desired state enable self-healing?

Hands-on exercises

1. Write a Dockerfile for a small web app and run it as a container.
2. Explain why a container starts far faster than a VM.
3. Describe what Kubernetes does when a pod crashes.
4. Decide whether a 2-service side project needs Kubernetes, and justify.

Mini project

CONTAINERIZE AND ORCHESTRATE

Containerize a small app with Docker and run it. Then deploy it to a local Kubernetes cluster with a Deployment (3 replicas) and a Service. Kill a pod and watch Kubernetes

recreate it; scale replicas up and down and observe the load spread. Feel declarative desired state and self-healing.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE RUNTIME SUBSTRATE

Containerize each service, define Kubernetes Deployments and Services with health checks and autoscaling (Chapter 10), and integrate with your service discovery (Chapter 30) and load balancing (Chapter 7). Decide replica counts from your Chapter 5 numbers, and keep state out of containers (in the datastores of Part III).

Chapter summary

- Containers package an app with its environment for **consistent, portable** runs, solving “works on my machine.”
- **VMs** run a full guest OS each (heavy); **containers** share the host kernel (light, fast, dense).
- **Docker images** are portable, immutable artifacts.
- **Kubernetes** orchestrates containers via **declarative desired state** — scheduling, scaling, and self-healing.